Build a specialized agent

A Service Desk assistant over three in-house systems

Stefano Schuppli

138 lines of Python, no framework. A SQLite accounting database, a live metrics service and a folder of runbooks — and a model that works out for itself which of them each question needs.

The scenario — a Service Desk assistant

One question from a user. The answer is spread across three systems, and none of them knows about the other two.

> "*Job 4831 failed on nid02 — what happened and what should I do?*"

To answer that, somebody normally opens three windows:

- the **accounting database** — what state did the job end in, what did it cost
- the **monitoring** — is that node healthy *right now*
- the **runbook** — what is the procedure when it is not

That shape — history, live state, procedure — is most of what a service desk does. It is why this makes a good first agent.

What it can reach

Three sources, and deliberately three different kinds of integration.

accounting.db

SQLite · `run_sql`

240 jobs, 12 users, 5 projects.

What happened.

telemetry

HTTP :8088 · `get_node_metrics` `list_alerts`

Temperature, power, alerts.

What is happening now.

docs/

files · `search_docs`

Four runbooks and policies.

What to do about it.

A database driver, an HTTP call and a file read — because that is what you actually have in-house. Four tools over three sources: one source can expose several.

1 — The database

A miniature Slurm accounting database, standing in for whatever your unit really queries.

```
CREATE TABLE jobs (
    job_id INTEGER PRIMARY KEY, username TEXT, project_id TEXT,
    partition   TEXT,      -- normal | gpu | debug | prepost
    nodes INTEGER, node_list TEXT,   -- 'nid02' or 'nid[02-05]'
    state       TEXT,      -- COMPLETED | FAILED | TIMEOUT | OUT_OF_MEMORY
    exit_code INTEGER, submit_time TEXT, start_time TEXT, end_time TEXT,
    node_hours  REAL);     -- nodes x elapsed hours
```

…plus `projects` (allocation in node hours, PI) and `users`.

> The schema is read back out of `sqlite_master` at start-up and pasted into the system prompt. **The model writes every query itself** — there is not one SQL statement written in advance anywhere in the application.

2 — The metrics service

A separate process, so the agent has to reach it over HTTP exactly as it would reach the real monitoring API.

```
GET /nodes            all eight nodes
GET /metrics/<node>   temperature, power,
                      GPU util, throttling
GET /alerts           what is firing now
```

```
{ "node": "nid02",
  "temp_c": 88.5,
  "power_w": 581,
  "throttling": true }
```

Values drift on every call — `curl` it twice and they change. **nid02 has been above the 85 °C throttle point for three days**, and that single fault is what the whole demo hangs on.

3 — The runbooks

Prose, not data. The part that says what to *do*.

`runbook-thermal-throttling.md` hot node → 30-50% slower → TIMEOUT
`runbook-out-of-memory.md` exit 137, and when it is the node's fault
`policy-fair-share.md` allocations, the 80% warning, refunds
`policy-maintenance-windows.md` drains, scheduled and not

`search_docs` is a keyword grep in one line of Python. Crude on purpose — and it misses often enough that you get to watch the agent read the failure and try a better word.

`ask.py` — 138 lines, four blocks

That is the whole application. There is no framework underneath it.

1 — The tools

Four ordinary Python functions. Nothing in them knows that an LLM exists.

2 — The tool list

The JSON schema the model is shown. Sent on every request.

3 — The system prompt

Who it is, today's date, and the database schema.

4 — The loop

send → read → execute → append → repeat.

The `openai` package is used **only as an HTTP client**. The agent is the code around it: the model reasons, this program acts.

The tool list — what the model is actually shown

Appended to every request, alongside the question. You never write it by hand and you never see it — here it is, for one tool.

```
{"type": "function", "function": {
  "name": "run_sql",
  "description": "Run one read-only SELECT against the Slurm accounting
                  database (tables: projects, users, jobs). Use it for
                  anything historical: who ran what, node hours consumed,
                  job states, allocations.",
  "parameters": {"type": "object", "required": ["query"], "properties": {
      "query": {"type": "string", "description": "A single SELECT statement."}}}}}
```

Four of these. **This is the entire extension mechanism:** to point the agent at your systems, write a function and describe it here. The description is prompt engineering — it is how the model decides when to reach for it.

The loop

Twenty lines. This is what makes it an agent rather than a chat window.

```
for turn in range(1, args.max_turns + 1):
    reply = client.chat.completions.create(model=args.model,
                                           messages=messages, tools=TOOLS)
    message = reply.choices[0].message
    messages.append(message.model_dump(exclude_none=True))

    if not message.tool_calls:                # nothing left to look up
        print(message.content); break

    for call in message.tool_calls:           # THE AGENT executes, not the model
        params = json.loads(call.function.arguments)
        result = DISPATCH[call.function.name](**params)
        messages.append({"role": "tool", "tool_call_id": call.id,
                         "content": result})
```

> The model never touches the database. It names a function and some arguments; `DISPATCH` decides whether that happens.

Start it

No virtualenv and no install step — `uv` fetches the two dependencies for the one command.

```
python3 seed.py                # once: builds accounting.db, 240 jobs, deterministic
python3 metrics_service.py     # second terminal — leave it running

uv run --with openai,httpx ask.py "which project is closest to its allocation?"
```

Key — `$CSCS_INFERENCE_API_KEY`, or `~/agent-sandbox/secrets/cscs-api-key`, created at `ui.inference.cscs.ch`.

Model — `moonshotai/Kimi-K2.7-Code` on `api.inference.cscs.ch`. `--model` to try another.

What you see while it runs

The log is half the point: every tool call, its arguments, and the context growing underneath.

```
── turn 1 ──────────────────────────────── 628 tokens sent (0 cached)
  ⚙ run_sql(query='SELECT * FROM jobs WHERE job_id = 4831')
    ← 288 chars: [{"job_id": 4831, "username": "mrossi", "state": "TIME…
  ⚙ get_node_metrics(node='nid02')
    ← 201 chars: {"node": "nid02", "temp_c": 88.5, "throttling": true, …
  ⚙ search_docs(keyword='TIMEOUT')
    ← 1,261 chars: ### runbook-thermal-throttling.md # Runbook: node runn…

── turn 2 ──────────────────────────────── 1,527 tokens sent (576 cached)
```

628 → 1 527 → 2 134 tokens sent. The model keeps no memory between calls, so every turn re-sends the whole conversation. That growth *is* the running cost of an agent. The `cached` figure beside it is the endpoint's prompt cache taking the sting out of it.

Prompts to try YOUR TURN

Same binary every time. Nothing in the code branches on the question.

`"which project is closest to using up its allocation?"`	run_sql alone
`"what is our policy on refunding node hours lost to a hardware fault?"`	search_docs alone
`"is anything wrong with the cluster right now?"`	list_alerts alone
`"which nodes have had the most TIMEOUT jobs since 1 September?"`	run_sql, one GROUP BY it wrote itself
`"job 4831 failed on nid02 — what happened and what should I do?"`	**all three sources**

The last one is the one to sit with: four tool calls in the first turn, then one answer — the job burned its walltime because nid02 is throttling, here is the runbook, and the 96 node hours are refundable.

The rule that is actually enforced

Ask it to drop a table and it declines politely. That politeness is not a control.

```
def run_sql(query):
    q = query.strip().rstrip(";")
    if not q.lower().startswith("select") or ";" in q:
        return "REFUSED: this tool runs a single SELECT and nothing else."
```

The model's refusal

Prose weighed against everything else in the context. Usually holds. Never checked.

The three lines above

Run before the query does, on every call, whatever the model asked for.

The database is also opened `mode=ro`. A rule written in prose is a suggestion; a rule written in code is a control.

Take it apart YOUR TURN

Try

- add a fifth tool against a system *you* own
- delete `search_docs` and watch the answers get vaguer but no less confident
- `--model zai-org/GLM-5.2` — a different model reaches for different sources
- remove the `SELECT` guard, then re-run the injection

What it deliberately is not

- no memory between runs — every question is a fresh context
- no permission prompts: every tool is allowed, because every tool is small
- a keyword grep, not retrieval
- one user, one machine, no auth

Over to you

A tool is a function. The tool list is its documentation.

Everything else in this application is code you already know how to write.